



AgentFirewall

Proof-backed execution firewall for autonomous EVM agents

BUIDL CTC 2026 Fall · Whitepaper

Attestcoin / USC · Creditcoin · Guarded execution

Executive summary

AgentFirewall is an execution firewall for autonomous EVM agents. It separates the ability to **propose** a transaction from the authority to **sign** it. Before a transaction reaches the signer, AgentFirewall requires a proof-backed authorization published on a source chain, verified through Attestcoin and recorded semantically on Creditcoin, plus an independent controller approval and exact execution matching.

The security claim is narrow and testable: a candidate can simulate successfully and still be rejected before `signTransaction()` if it does not match the verified authorization. The repository contains a deterministic judge harness and testnet reference evidence for both the blocked and allowed paths.

The problem

Autonomous transaction systems typically combine several components: a model or planner, tools, RPCs, simulation, policy logic and a wallet. Any component before the wallet can be compromised or simply wrong. Simulation catches execution failure; it does not prove that the human or application authorized the exact recipient, target, calldata, value, nonce, runtime code identity and fee envelope that is about to be signed.

Giving an agent a raw signer therefore collapses two distinct powers into one process: proposing an action and authorizing money movement. AgentFirewall keeps those powers separate.

Architecture



The Creditcoin registry does not accept application JSON as truth. It calls the native BlockProver at `0x0000000000000000000000000000000000000000000000000000000000000000FD2`, decodes the proven source transaction and records the authorization semantics derived from it.

The runtime proof client uses the Creditcoin Proof Builder HTTP endpoint to obtain the proof and independently preflights it against the native BlockProver before submitting the semantic registry transaction. There is no undeclared runtime SDK dependency.

What is bound

The authorization/execution path binds the security-relevant fields needed to stop common agent mutations: execution chain, sender, execution nonce, expiry, action, target, target runtime-code hash, asset, counterparties, amount/flag, calldata hash and native value. The controller approval additionally binds the authorization identity and execution context. Local hard-policy/analyzer denials remain an independent deny layer.

Product and users

The initial users are infrastructure teams that let autonomous software prepare EVM transactions but cannot safely give that software unconditional signing authority:

- agent-wallet and wallet-infrastructure teams;
- MCP and agent-framework maintainers;
- automated treasury operators;
- DeFi automation and transaction-bot operators;
- applications that need delegated execution with an auditable authorization source.

This repository does **not** claim production adoption or paid users. The market thesis is instead concrete and falsifiable: as agents gain transaction authority, wallet teams need a boundary that verifies *authorization*, not merely executability or model intent. Creditcoin can serve that boundary even when the resulting business transaction executes elsewhere, expanding its useful surface from destination-chain activity to cross-chain agent authorization infrastructure.

Why Creditcoin and Attestcoin are load-bearing

Creditcoin is used as a cross-chain verification and authorization plane. Attestcoin proves the source-chain authorization transaction; Creditcoin converts that proof into a registry record consumed by the signer boundary. The agent's eventual execution may occur on another EVM chain, which lets Creditcoin secure agent activity beyond applications whose final transaction is itself on Creditcoin.

Remove Attestcoin/Creditcoin and AgentFirewall loses the cryptographic link between a foreign-chain authorization and the signer's local decision. Replacing it with an application server or conventional oracle would change the trust model, not merely the implementation.

AI-track fit

The AI integration is not a chat wrapper. `src/mcp-server.ts` exposes tools that allow an agent to submit a declared intent and candidate transaction. The agent never receives the raw signer. Verified cross-chain data and deterministic policy decide whether the proposed action can reach `signTransaction()`, and an allowed proposal can trigger the on-chain execution.

That makes the AI component useful but non-authoritative: the model proposes; proof-backed policy authorizes.

Evidence

Bundled testnet reference evidence includes:

- Sepolia authorization transaction: `0xb128ed2d774c524d563d43df63349fd8ea6c39742a36a7f9a24d1d8309dab6aa`
- Creditcoin semantic ingest: `0x220f64a587451af1c425be74e644366b7784b9c26a7589cbfa412e4e69528062`
- guarded Sepolia execution: `0x16096b8bd03603994630c04afe09c1c25d2382e30338058c35d2f1333eb3feb7`

Observed security result:

mutated candidate	simulation=SUCCESS	decision=BLOCK	signerCallCount=0	
canonical candidate	simulation=SUCCESS	decision=ALLOW	signerCallCount=1	mined

A separate testnet run exercised the MCP stdio execution path. `npm run judge:verify` re-executes current GuardedSigner logic instead of trusting stored evidence.

Differentiation

AgentFirewall is not merely:

- an EVM simulator — both judge candidates can simulate successfully;
- a model prompt or MCP permission dialog — model output is not the authority source;

- a multisig replacement — controller approval is only one input to a proof-backed execution boundary;
- a centralized cross-chain oracle — the source transaction is verified through Attestcoin/native Creditcoin verification.

The distinctive boundary is **proof-backed authorization immediately before signing**.

Security posture

The design is fail-closed around the signing boundary. It freezes the execution-critical envelope, enforces fee ceilings, verifies proof-backed authorization and controller approval, rechecks chain/nonce/expiry/runtime code, signs once, reparses the raw signed transaction and rejects any mismatch before broadcasting.

The repository also separates deterministic runtime verification from stored testnet evidence so a judge can distinguish what is being executed now from what was observed previously.

Originality and provenance

Development started on September 7, 2026. The initial snapshot is recorded in `PROJECT_BASELINE.txt`; the Attestcoin/Creditcoin authorization path, controller separation, live E2E verification, MCP integration and hardened signer boundary were subsequent additions in this build. This provenance statement is intentionally narrow and does not claim prior adoption or production deployment.

Scope and limitations

This is testnet/reference code, not production custody software and not an audited wallet product. Current limitations include one configured trusted RPC for each verified read path, no explicit authorization revocation/epoch and proxy detection that covers common layouts rather than every custom proxy. A production version should add independent RPC/quorum strategy, revocation semantics, custody/HSM integration and external audit.

Repository: github.com/itisfloris/AgentFirewall